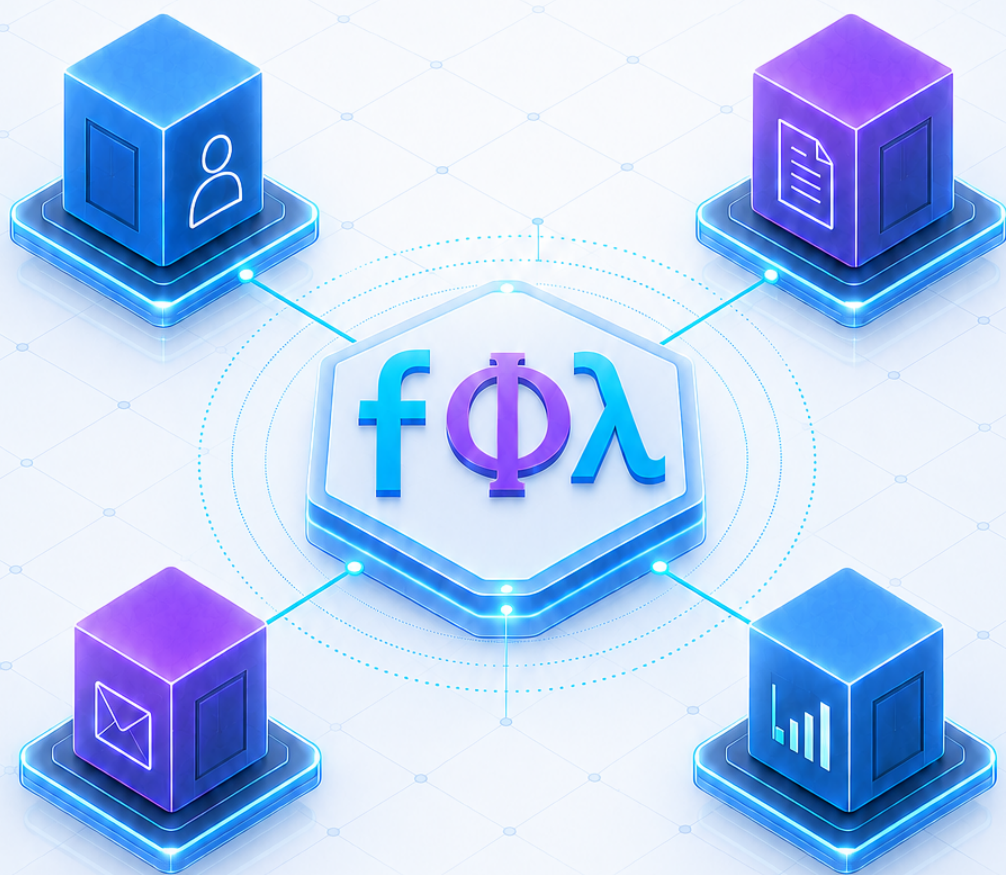


# Let's Build **Microservices** in Fo $\lambda$ ang



**Kameswara Rao Mithipati**

# Preface

## FROM LANGUAGE IDEAS TO WORKING SERVICES

Software concepts become meaningful when we use them to build something real. This book has a focused purpose: to guide you through designing and implementing microservices in FoLang, step by step, and then show how those services can work together as a practical distributed system.

The journey begins with a small request and response. From there, we gradually introduce routing, data modelling, validation, error handling, persistence, configuration, testing, security, observability, service boundaries, communication, resilience, and deployment. Each stage adds one useful capability while keeping the complete system understandable and runnable.

This is also an applied companion to **Think Like FoLang**, **FoLang in Depth**, and the **FoLang Language Reference**. The first develops the mental model and habits needed to express solutions naturally in FoLang. The second explores the language's concepts and mechanisms in greater depth. The reference defines the language precisely. This book brings those three perspectives into one continuous project and demonstrates where each idea becomes useful.

Rather than presenting isolated code fragments or hiding important decisions behind framework magic, the examples evolve deliberately. We will build, run, test, inspect, and refactor the services as their responsibilities become clearer. Along the way, we will discuss not only what works, but why a design fits FoLang and what trade-offs it introduces.

You need basic programming knowledge and a general understanding of HTTP, but you do not need previous microservices experience. Readers who are new to FoLang can learn through the progressive examples; experienced readers can use the same path to examine the language's approach to service design in a realistic setting.

By the end, the goal is not merely to have a collection of working services. You should be able to reason about them in FoLang, understand how their parts fit together, and confidently extend the system with services of your own.